Machine Learning for Named Entity Recognition and Classification Report, Theoretical Component, and NERC-Research Preparation

Vera Langeberg

Abstract

Fill in a short abstract for your final submission.

We recommend aiming for a report of around 6. This does not include acknowledgements or references (they may appear on page 7 and later if needed). You can use appendices if you want to share more details. Below, we provide length indications per section. Please try to stick to them, but don't worry about receiving a lower grade if you don't quite manage. They are supposed to help you and prevent you from doing extra work.

1 Introduction

Length: ± 1/2 - max 1 column for Introduction

Write a short introduction to your approach for the final submission. The introduction should include:

- A brief description of the task (not detailed)
- An outline of the ML approaches included in the report
- A brief summary of your results

Assignment 3

2 Task and Data

Length: ± 1.5-2 columns for Task and Data.

2.1 Task

The exact task of NER given the CoNLL dataset is to identify and classify named entity spans in text into predefined categories. These categories are: locations (LOC), organizations (ORG), miscellaneous names (MISC), and persons (PER). Named entities are typically spans of text that can consist of one or more tokens. However, to simplify the task and transform it from span identification into a token classification problem, the BIO (Beginning, Inside, Outside) tagging scheme is used. This scheme allows us to represent multi-token entity spans while still classifying individual tokens. Before them, there is a B- or I- to indicate whether it is the beginning or inside of an entity. The O label indicates that the token is outside of any named entity. This results in the following labels present in the dataset: "'B-LOC', 'I-ORG', 'I-MISC', 'B-MISC', 'O', 'B-ORG', 'I-PER', 'I-LOC', 'B-PER'"

For example, in the phrase "Vera Langeberg lives in Amsterdam":

The span "Vera Langeberg" is a person entity: "Vera" would be labeled as "B-PER" (beginning of person span) "Langeberg" would be labeled as "I-PER" (inside of person span) "lives" and "in" would be labeled as "O" (outside any entity span) The span "Amsterdam" is a location entity: "Amsterdam" would be labeled as "B-LOC" (beginning of location span) This approach allows the NER system to identify both single-token and multi-token named entity spans while performing token-level classification, effectively bridging the gap between span identification and token classification.

Assignment 3 Update if necessary

2.2 Dataset and Distribution

The dataset used in this project is the CoNLL-2003 dataset, which is specifically designed for Named Entity Recognition (NER) tasks (Sang and Meulder, 2003). The CoNLL format consists of for columns separated by spaces. Empty rows indicate sentence boundaries. The columns are as follows:

- Word or token

- Part-of-speech tag
- Chunk tag
- Named Entity tag

(Sang and Meulder, 2003)[p.2]

For example, in the line:

```
LONDON NNP B-NP B-LOC
```

"London" is the token, "NNP" is the part-of-speech tag (indicating a proper noun), "B-NP" is the chunk tag (indicating that it is the beginning of a noun phrase), and "B-LOC" is the named entity tag (indicating that it is the beginning of a location entity). The total number of tokens in the CoNLL training set is 203621. As can be seen in 1, the majority of the tokens are labeled with the O tag, indicating that they are outside of any named entity. The most frequent named entity label is B-LOC (7140), followed by B-PER (6600) and B-ORG (6321), as shown in 2. The least represented named entity label is I-MISC together with I-LOC. They only have 1155 and 1157 occurrences respectively in the training set.

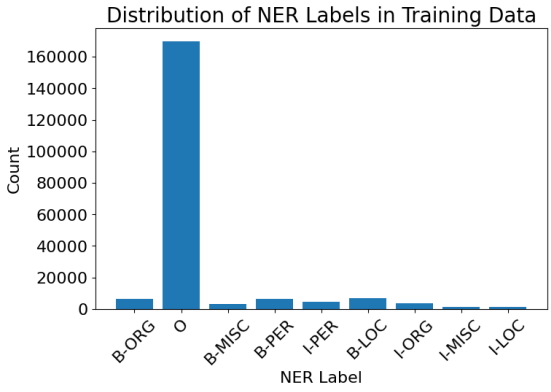

Figure 1: Label distribution in the CoNLL training set.

In 9, the most common words for location, persons, organizations and miscellaneous names are shown. Here the IOB tags have been removed for clarity. For location, "u.s" is most common, "cup" is most common for miscellaneous names,"clinton" for persons and "of" for organizations.

Assignment 3 update if necessary

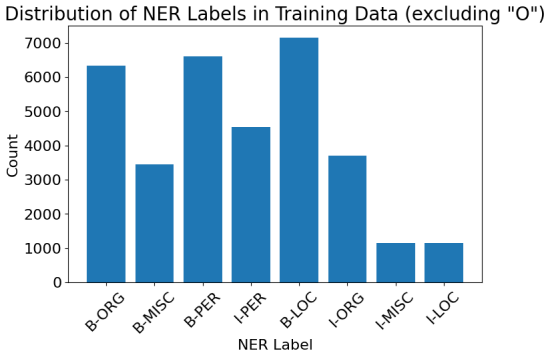

Figure 2: Label distribution in the CoNLL training set (excluding "O").

2.3 Preprocessing

The only preprocessing that was done for the initial dataset analysis was lowercasing the tokens for the most frequent words per category. This was only done to collect the most frequent words, to avoid having the same word in different cases. Since, the case of the token is an important feature for NER, the original casing was kept for the model training. No stopword removal or stemming/lemmatization was applied. Also no special characters were removed yet. This might be done in later assignments.

Assignment 3 update (if applicable)

2.4 Evaluation Metrics

This paper will focus on token-level evaluation for the assignments. This means, that the BIO tags will be taken into account when calculating the metrics and have to match exactly. If only a partial match is made (so either wrong BIO tag, or wrong NE label), it will be considered incorrect.

Assignment 3: Update if necessary

3 Models and Features

Length: ± 2 columns for Models and Features.

3.1 Models

Assignment 1 The model used in the basic system is a Logistic Regression classifier from the sklearn library. Despite its name, it is mostly used for classification tasks. It has a linear model as its core that uses the sigmoid function to output probabilities for each class. The model coefficients represent the

importance of each feature for each class. The model was trained using the default parameters from sklearn on the training set.

Assignment 2 Alternative Methods (SVM, NB) The models that were used in addition to Logistic Regression are Support Vector Machines (SVM) and Naive Bayes (NB). SVM is a powerful classification algorithm that works by finding the optimal hyperplane that separates different classes in the feature space. It is particularly effective for high-dimensional data and can handle non-linear decision boundaries using kernel functions. In this assignment, the SVM model was implemented using the sklearn library with default parameters.

Naive Bayes is a probabilistic classifier based on Bayes' theorem, which assumes independence between features. It is computationally efficient and works well for text classification tasks, especially when the feature space is large. To use the sparse features effectively, a Bernoulli Naive Bayes (BernoulliNB) variant was chosen, because it is designed to handle binary/boolean features and copes well with one-hot encoded features. The NB model was also implemented using the sklearn library with default parameters.

Assignment 3 More advanced models (fine-tuning BERT, CRF for ReMA students only)

3.2 Features

Assignment 1 Feature exploration In this paper the features that were used in the basic system are:

- Token itself: The actual word provides crucial semantic information. Certain words are more likely to be part of specific entity types (e.g., "Inc." often indicates an organization).
- POS tags: Part-of-speech information helps identify potential named entities. For instance, proper nouns (NNP) are often good candidates for named entities.
- Capitalization: In English, this is a strong indicator for named entities, as proper nouns are typically capitalized. It helps distinguish between common nouns and potential named entities, though it can be less reliable at the beginning of sentences.
- Previous POS tag: POS tags order matter for the current token's POS tag, which matters for the NER label
- Presence of digits: Tokens containing digits are often part of named entities, such as dates, times, or numerical identifiers.
- Position in the sentence: The position of a token in the sentence can provide context for its meaning and help disambiguate its role in the entity.

These features provide complementary information that aids in identifying and classifying named entities. The token itself carries semantic content, POS tags offer syntactic clues, and capitalization provides orthographic hints that are particularly useful for recognizing proper nouns. The previous POS tag adds contextual information, while the presence of digits and position in the sentence help capture specific patterns associated with named entities.

The distribution of the first three features can be found in the 7.

Assignment 3 More advanced features

4 Experiments and Results

Length: ± 3 columns (including figures and tables) for Experiments and Results. Additional material can be provided in the Appendix.

4.1 Hyperparameter Tuning

For the SVM model, a grid search was performed to find the best hyperparameters. A randomized search was performed because it offered a good trade-off between exploration and computational efficiency. The datasplit was made by using cross fold validation with k=5 on the development set. Using cross fold validation helps robustness of the results, since the model is trained and evaluated on different subsets of the data. The hyperparameters that were tuned are:

- C: Regularization parameter. Sampled from a uniform distribution between 0.1 and 100.
- Kernel: Kernel type to be used in the algorithm. Values tested: ['linear', 'rbf', 'sigmoid']

- Gamma: Kernel coefficient for 'rbf' and 'sigmoid'. Sampled from a uniform distribution between 0.001 and 1.0.

On the test machine (8 cores), it took about 22 hours to complete the randomized search with 5 iterations and 5 fold cross validation. The best hyperparameters found were:

- C: 15.699452033620265
- Kernel: linear
- Gamma: 0.05908361216819946 (not used by linear kernel, but kept for reference)

After hyperparameter tuning, the SVM model's performance improved significantly. Especially on the I-MISC and I-LOC labels, where previously no correct predictions were made. The results are shown in the table 4 and the confusion matrix 11.

4.2 Evaluation

Results updated The updated results of the basic system on the development set are shown in figure 1 and the confusion matrix 3.

Table 1: NER classification report for Logistic Regression

Tag	Prec.	Rec.	F1	support
B-LOC	0.853	0.846	0.850	1837
B-MISC	0.907	0.747	0.819	922
B-ORG	0.713	0.775	0.743	1341
B-PER	0.920	0.680	0.782	1842
I-LOC	0.778	0.708	0.741	257
I-MISC	0.800	0.613	0.694	346
I-ORG	0.779	0.565	0.655	751
I-PER	0.777	0.930	0.847	1307
O	0.982	0.996	0.989	42759
accuracy			0.957	51362
macro avg	0.834	0.762	0.791	51362
weighted avg	0.956	0.957	0.955	51362

The results of SVM and NB on the development set are shown in figure 3 and 5 as well as the confusion matrices 10 and 12.

An interesting observation is that the "O" label gets predicted very well by all models, with precision and recall values above 0.96. This is likely due to the high frequency of the "O" label in the dataset, making it easier for the models to learn to predict it accurately. On the other hand, labels like I-LOC and I-MISC have very low precision and recall across all models, indicating that these labels are more challenging to predict accurately. This could be due to the lower frequency of these labels in the dataset, making it harder for the models to learn their patterns.

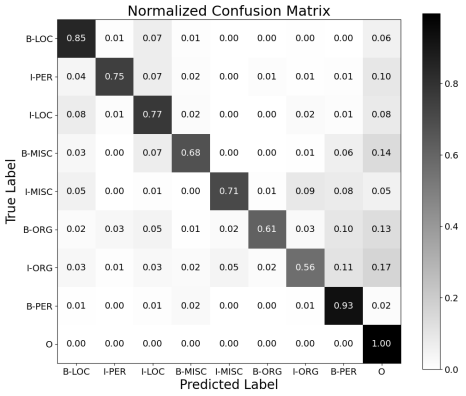

Figure 3: Confusion matrix for Logistic Regression model.

Naive Bayes is very bad in the I-MISC and I-LOC labels, since it never predicts them. Logistic Regression performs best overall, with the highest accuracy (0.957), followed by Naive Bayes (0.912) and SVM (0.893).

Assignment 3: update

4.3 Feature Ablation

The feature ablation was done based on the linear SVM model, with no hyperparameter tuning. This decision was made based on time constraints, since the hyperparameter tuning took a lot of time. Also there were over twenty configurations of features to test, which already made training take up a lot of time. In order to display which features have proven to be the most useful, a lot of configurations of feature combinations were tested. The results of these combinations are shown in table 7. The most valuable feature seems to be the token itself, since removing it causes a big drop in performance. The previous part of speech tag, also has a good impact on the performance, since removing it causes a drop of 2 F1 points. A lot less impact is made by digits and case. It seems that without word embedding or token, the results are dropping.

5 Error Analysis

Length: ± 1.5-2 columns for Error Analysis.

Assignment 3: beyond the confusion matrix

6 Discussion

Length: ± 1 column for Discussion and Conclusion together.

Assignment 3

7 Conclusion

Assignment 3

Acknowledgements

References

Heeyoung Lee, Angel Chang, Yves Peirsman, Nathanael Chambers, Mihai Surdeanu, and Dan Jurafsky. 2013. Deterministic coreference resolution based on entity-centric, precision-ranked rules. *Computational Linguistics*, 39(4):885–916.

David Nadeau and Satoshi Sekine. 2007. A survey of named entity recognition and classification. *Lingvisticae Investigationes*, 30:3–26.

Preslav Nakov, Zornitsa Kozareva, Alan Ritter, Sara Rosenthal, Veselin Stoyanov, and Theresa Wilson. 2019. Semeval-2013 task 2: Sentiment analysis in twitter. *CoRR*, abs/1912.06806.

Erik F. Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition.

Warto, Supriadi Rustad, Guruh Fajar Shidik, Edi Noersasongko, Purwanto, Muljono, and De Rosal Ignatius Moses Setiadi. 2024. Systematic literature review on named entity recognition: Approach, method, and application. *Statistics, Optimization & Information Computing*.

Theoretical Component

Assignment 1 - What is Machine learning?

My answer on the question "What is Machine learning" starts with explaining the intuition behind it. Machine learning is about machines learning, but this sounds still very silly and unclear. Therefore, lets first dive into the machines part. People should not think about machines like the first typewriter, or motor engines, or even your normal kitchen blender. We should also not think about the computers or calculators that run simple algorithms. Algorithms are mostly strictly defined, like the ones embedded in your regular calculators. These machines do not learn, they just follow a set of rules that are already defined. This is not machine learning, it is following a recipe. If the recipe is bad, you will get a burned cake. If you don't adjust the oven temperature next time, the cake will burn over and over again. The machines we are looking at are different. They can learn from data, identify patterns, and make decisions based on that data To get back to our cake, this means that after one time burning the cake, it will lower the temperature next time, or even adjust the baking time. It can be undercooked that time, but it will learn from that as well, and adjust again. This learning from data and identifying patterns is the essence of machine learning.

When looking at it in a bit more formal way, machine learning is a subset of artificial intelligence (AI) that focuses on the development of algorithms and statistical models that enable computers to perform specific tasks without explicit instructions. Instead of being explicitly programmed to perform a task, machine learning algorithms use data to learn patterns and make predictions or decisions. This is done by training the algorithms on large datasets, allowing them to learn from the data and improve their performance over time. Machine learning can be categorized into three main types: supervised learning, unsupervised learning, and reinforcement learning. In supervised learning, the algorithm is trained on labeled data, where the input data is paired with the correct output. The algorithm learns to map the input to the output and can make predictions on new, unseen data. In unsupervised learning, the algorithm is trained on unlabeled data and must find patterns and relationships in the data on its own. Reinforcement learning involves training an agent to make decisions in an environment by rewarding or punishing it based on its actions. Key in the development of good machine learning algorithms are examples, features, and labels. Examples are the individual data points that the algorithm learns from. Features are the attributes or characteristics of the examples that are used to make predictions. Labels are the correct outputs or categories that the algorithm is trying to predict. To test the performance of machine learning algorithms, evaluation metrics such as accuracy, precision, recall, and F1-score are used. The evaluation is typically done on a separate dataset that was not used during training, known as the test set. A golden testset is a testset that is carefully curated and annotated by experts to ensure high quality and reliability.

Assignment 2

1.A.1: Task definition

A task definition of Subtask B (Nakov et al., 2019) with regards to input and desired output can be formulated as follows. Create a system that, given a message (single, unprocessed text message) from Twitter as input, decide the dominant sentiment from positive, negative, or neutral. For messages conveying both a positive and a negative sentiment, whichever is the stronger one was to be chosen. If no clear sentiment is present, neutral should be chosen. The output of the system are the predicted sentiment labels corresponding to the input messages.

1.A.2: Key Features for Sentiment Analysis in Tweets

To successfully determine the overall sentiment (positive, negative, or neutral) of a social media message, a system can rely on several types of numerical features that capture both the literal meaning of the words and the highly expressive style of online communication. The most fundamental features come from sentiment lexicons and N-grams. Lexicon-based features involve simply counting how many words in the tweet are listed in precompiled dictionaries as inherently positive (like "awesome") or negative (like "terrible"). This gives the system a strong initial signal of the message's overall leanings. N-grams, on the other hand, look at sequences of words (like "not nice" or "hate it") to capture context. This is crucial for recognizing phrases where one word negates the sentiment of another, ensuring the system doesn't misclassify "not nice" as a positive expression.

Beyond the words themselves, social media posts frequently use special elements to intensify

emotion. Features that track punctuation and emoticons can be highly valuable. Users often signal strong feelings using multiple exclamation points ("!!!") or specific emoticons like ":D" for joy or ":(" for sadness. Similarly, elongation and capitalization act as indicators of emphasis or anger. For example, a system can easily detect heightened emotion when a word is spelled with repeated letters ("sooo") or written entirely in capital letters ("SO ANGRY").

Finally, the system should incorporate features specific to the social media platforms. These can be hashtags or @-symbols that indicate towards which named entity the message is directed.

1.A.3: Feature representations

For sentiment analysis to get the desired outputs it starts by converting a text message into a Feature Vector ($\mathbf{x}$) within a Vector Space Model (VSM). The system first defines a large vocabulary of possible features, such as specific unigrams, the bigram "not nice," or the binary feature Has Positive Emoticon. Every message is then represented by an array of numbers where each component, x_j, records the magnitude or presence (e.g., count or a binary 1/0) of the corresponding feature j in that specific Tweet. For capitalization it would be usefull to look at the ratio of capitalized words compared to not capitalized. This would also be a numerical representation. Important, the computer system sees only will see a high-dimensional, probably sparse (mostly zero-filled), numerical vector, not the human-readable text. The machine learning classifier, such as a Support Vector Machine (SVM), then utilizes these vectors to learn sentiment. During training, the classifier assigns a weight (w_j) to every feature j, with the weight indicating the feature's predictive power. For instance, the feature Count of Positive Words will receive a high positive weight, while the feature Bigram: "not nice" will receive a significant negative weight, reflecting its counter-intuitive sentiment. When classifying a new, unseen message, the system calculates a score by taking the linear combination of the feature vector and the learned weights, Score $= \sum_j w_j x_j$. This numerical score is then compared against established thresholds to assign the final class (positive, negative, or neutral). This mechanism reveals the core difference between human and machine perception. The human uses linguistic insight to understand the message's meaning, context, and intent (e.g., "This message is positive because the word 'brilliant' is a strong compliment"). In contrast, the computer only performs mathematical algebra on the numbers, learning that the presence of the feature associated with "brilliant" has a specific high positive weight, which pushes the aggregate score above the positive threshold. The system's success is therefore entirely dependent on the quality of the initial human-engineered features that capture the necessary linguistic cues, so this part of the assignment is very important to me.

The machine's interaction with the feature vector ($\mathbf{x}$) reveals its non-linguistic approach to complex phenomena. The system does not "see" negation or composition in a semantic sense; instead, it relies on the algebraic combination of specific, engineered features. For instance, a human understands that the word "not" reverses the meaning of "good." The machine, however, only registers the co-occurrence of the feature Unigram: "good" (which has a high positive weight, w_{good}) and the feature Bigram: "not good" (which has a corresponding high negative weight, $w_{\text{"not good"}}$). The sentiment reversal is a mathematical consequence of the learned weights, not a linguistic understanding of the negation's scope. The use of n-grams, like bigrams, is the machine's primary proxy for approximating scope and composition, as it links adjacent words to capture local context (e.g., distinguishing "New York" from "New" + "shoes"). While beneficial for capturing composition, including bigrams vastly increases the size of the feature vocabulary. This leads to a higher-dimensional vector space that is highly sparse. This means the vector $\mathbf{x}$ contains far more zero-entries. This increased sparsity poses a challenge for the classifier, as it makes it harder to gather enough training examples for each specific n-gram feature to learn a reliable, non-zero weight. Also training time will increase, because the model has to process many more features.

1.B.1: Useful features according to rule based system

Central to the paper of (Lee et al., 2013) is a deterministic coreference solution that starts with mention detection. In the mention detection phase, nominal and pronominal mentions are identified using a high recall algorithm. The coreference stages follows and applies 10 different models "sieves" from highest to lowest precision. The highest precision model is the speaker identification sieve, which is

highly useful to connect first and second person pronouns inside quotation marks to the nearest recognized speaker. This would result in True for the pair (I, Vera) in the text: "I can't wait to see it", Vera said. Another useful model is the exact string match sieve, which links mentions with identical strings. This could be done through a head match like "company" in "the tech company". or precise like "VU" in "The best university in Amsterdam is the VU". They have extremely high precision and can therefore establish coreference links with high confidence. The final feature I want to highlight is the pronominal agreement constraint. Which is a binary feature that checks for agreement between a pronoun and possible antecedent. It checks if there is number agreement in the pair (singular vs plural) and gender agreement (male, female, neutral), and animacy agreement (animate vs inanimate). For example, in the sentence "Vera walks her dog everyday, she loves it" the pronoun "she" agrees in number and gender with "Vera", making it a likely antecedent.

1.B.2: Present the features to the system

In this section I discuss two of the features mentioned above in 1.B.1 and explain how to represent this feature to a system and how it differs, from what humans see. **Exact String Match Sieve** The exact string match sieve is represented by a set of simple, binary features based on the surface form of the mentions. For any candidate mention pair (M_i, M_j), the system calculates: $\text{Is\_Exact\_String\_Match}(M_i, M_j)$ (True/False) and $\text{Is\_Head\_Word\_Match}(M_i, M_j)$ (True/False). These features offer the computer the highest degree of certainty because they rely on deterministic character-by-character comparison. What the computer sees is a calculation result like $\text{Head\_Match} = 1$. This contrasts sharply with what a human sees: we instantly recognize the identity of two mentions like "the President" and "President" as a semantic guarantee. The computer, lacking semantics, uses the head-word match as a heavy weight feature.

Pronominal Agreement The Pronominal Agreement feature, presents a more difficult challenge as it requires complex inference and aggregation. The computer must first assign linguistic attributes (Gender, Number, Animacy) to the candidate pronoun ($M_{pronoun}$) based on static lists. Next, it must check these against the entire antecedent cluster ($E_{antecedent}$). $E_{antecedent}$ is not a single piece of text; it's a dynamic data structure storing the consensus attributes inferred from all previous mentions linked to the entity. The system then generates binary constraint features like: $\text{Number\_Violation}(M_{\text{pronoun}}, E_{\text{antecedent}})$ and $\text{Gender\_Violation}(M_{\text{pronoun}}, E_{\text{antecedent}})$. What the computer sees are these violation flags. If Gender_Violation is True, the classifier assigns a massive negative weight to the potential coreference link. What a human sees is an immediate semantic and real-world absurdity (e.g., trying to refer to a table with the pronoun "he"). The computer approximates this human constraint through a cascade of fallible preprocessing steps (NER to determine animacy, part-of-speech tagging to determine number) followed by a simple binary comparison.

Assignment 3 - K-nearest neighbours

A KNN classifier works as follows: for each new datapoint, it looks at the k (parameter that you can tune) closest datapoints that were already classified. It assigns the label to the new datapoint that is most common among those k neighbours.

To illustrate this working, the new datapoint in the figure in the assignment forms an example. There are two green triangles surrounding our new datapoint and only one yellow square, so it would assign a green triangle in case k=3 (like shown in the figure). If k=number of total datapoints in the figure, the new label would get assigned a yellow square. This is because there are more yellow squares in the figure, than green triangles. If k=1, the new datapoint would get assigned a yellow square (the closest neighbour seems to be yellow, altough it was hard to see clearly in the figure).

The result of picking a very small k, is a bigger risk op overfitting. The model becomes very sensitive for noise, because the outlier datapoints are weighing quite heavy. If k is set very large, it will always predict the most common label in the dataset, and it will not be sensitive to any patterns in the data. Its scope will be so broad, that it will fail to detect any boundaries, or patterns, resulting in an underfitted model.

Time spent

Week	Task	Time
1	Understand the assignment and getting everything setup	30 minutes
1	Watch the videos of module 1	2 hours
1	Finish 2.1 about understanding NER and CoNLL	1.5 hours
1	Work on 2.2.1 further exploration of the data	1 hour
1	Work on 2.2.2 feature hypothesis	1 hour
1	Started with 3.1, description of precision, recall etc	0.5 hour
2	Did all the week 2 tasks for assignment 1	2 hours
2	Incorporated the feedback for assignment 1	1.5 hours
3	Worked on the theoretical step of assignment 2	1 hour
3	Did a brief literature study for efficient features	1.5 hours
3	Implemented the features from the literature study	2.5 hours
3	Trained the models SVM and NB	1.5 hours
3	Evaluated the models and created confusion matrices	1.5 hours
3	Updated the report for assignment 2	2 hours
4	Worked on 1.B theoretical part	1 hour
4	Worked on assignment 2 word embeddings	3 hours
4	Worked on assignment 2 hyperparameter tuning	6 hours (training and tuning took about 26h in total)
4	Updated the report for assignment 2	2 hours
5	Incorporated feedback for assignment 2 theoretical	0.5 hour
5	Did the theoretical part of assignment 3 KNN	0.5 hour
5	Programmed the feature ablation for 5 combinations	1 hours
5	Trained and evaluated the feature ablation models	2.5 hours (run time)
5	Updated the feature ablation with about 15 new configurations	0.5 hours
5	Let the new feature ablation run	6 hours (run time)
5	Updated feature ablation text for assignment 3	2 hours
5	Started studying for the final exam	5 hours
5	Updated feature ablation configs again	1 hour (2 hours runtime)
5	Updated the report again based on new feature ablation results	1 hours
Total	43 hours	

Table 2: Time overview.

NERC-research preparation

2.1.1 - NER given the CoNLL dataset

In this course we will use the CoNLL-2003 dataset, which is specifically designed for Named Entity Recognition (NER) tasks (Sang and Meulder, 2003). The exact task of NER given the CoNLL dataset is to identify and classify named entities in text into predefined categories. These categories are: locations (LOC), organizations (ORG), miscellaneous names (MISC), and persons (PER). Before them, there is a B- or I- to indicate whether it is the beginning or inside of an entity. The O label indicates that the token is outside of any named entity. This results in the following labels present in the dataset: "'B-LOC', 'I-ORG', 'I-MISC', 'B-MISC', 'O', 'B-ORG', 'I-PER', 'I-LOC', 'B-PER'"

The conventions that are followed in this dataset is IOB tagging scheme. This means that when a named entity consists of multiple tokens, the first token is labeled with a B- (beginning) and the subsequent tokens with I- (inside). For example, in the entity "New York City", "New" would be labeled as B-LOC, "York" as I-LOC, and "City" as I-LOC. There is consistency in the labeling, meaning that if a token is labeled as I-ORG, it must be preceded by either B-ORG or another I-ORG.

Punctuation marks are represented as separate tokens and are labeled with the O tag, indicating that they are outside of any named entity.

2.2.1 - How does CoNLL work?

The CoNLL format consists of for columns separated by spaces. Empty rows indicate sentence boundaries. The columns are as follows:

- Word or token
- Part-of-speech tag
- Chunk tag
- Named Entity tag

(Sang and Meulder, 2003)[p.2]

For example, in the line:

```
LONDON NNP B-NP B-LOC
```

"London" is the token, "NNP" is the part-of-speech tag (indicating a proper noun), "B-NP" is the chunk tag (indicating that it is the beginning of a noun phrase), and "B-LOC" is the named entity tag (indicating that it is the beginning of a location entity).

2.2.1 - Distribution of the labels

To investigate the distribution of the labels in the CoNLL dataset, I wrote a small piece of code that extracts the labels from the dataset and counts their frequency. The total number of tokens in the training set is 203621. The results show that the majority of the tokens are labeled with the O tag, indicating that they are outside of any named entity. This was what I expected, as not all tokens in a text are part of named entities. When looking at the named entity labels, I found that the most frequent named entity label is B-LOC (7140), followed by B-PER (6600) and B-ORG (6321). The least represented named entity label is I-MISC together with I-LOC. They only have 1155 and 1157 occurrences respectively in the training set.

2.2.2 - Hypothesis about linguistic features that might help NER

I hypothesize that the following linguistic and orthographic features might help improve NER performance: - Part-of-speech tags: Certain parts of speech, such as proper nouns (NNP), are more likely to be named entities. Including POS tags as features can help the model identify potential named entities. - Capitalization: Named entities often start with capital letters. Including a feature that indicates whether a token is capitalized can help the model distinguish named entities from common nouns. - Contextual

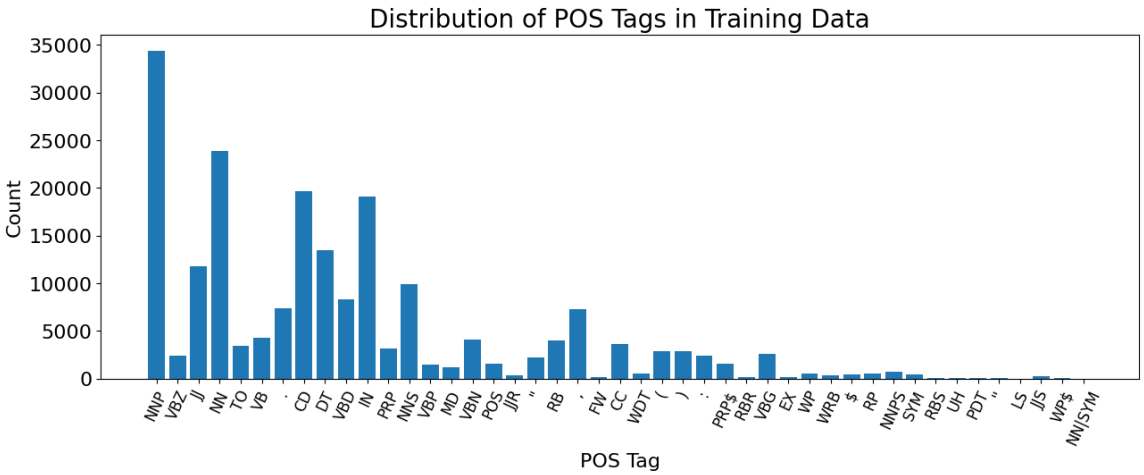

Figure 4: Distribution of POS tags in the CoNLL training set.

words: The words surrounding a token can provide important context for identifying named entities. "the" or "an" or "a" is very common before a named entity. Including features that capture these patterns can help the model recognize named entities.

To test if these features have influence on the NER performance, I wrote some code in the notebook. Starting with the POS tags. I extracted the POS tags from the dataset and counted their frequency. The results show that the most common POS tags in the CoNLL training set are NNP 4. Next, I looked at the capitalization feature. I counted the number of capitalized tokens versus non-capitalized tokens. The results show that a significant (about 25%) portion of the tokens in the CoNLL training set are capitalized 5. Finally, I analyzed the contextual words preceding named entities. I extracted the tokens that appear before named entities and counted their frequency. The results show that certain tokens, such as "the", "of", and "in", are more common before named entities 6 and 7. This didn't correspond exactly with my hypothesis, as I expected more articles like "a" and "an" to be present.

Next, I explored the most common words per category. Since the figure 8 is a bit hard to read, I created a new graph without the IOB tags. This graph shows the most common words for location, persons, organizations and miscellaneous names. 9. For location, "u.s" is most common, "cup" is most common for miscellaneous names,"clinton" for persons and "of" for organizations.

3. 1. Confusion matrix

In this section I discuss the terms precision, recall and F-score in my own words. The precision of a model is the proportion of true predictions compared to all instances that got the true label. So how much that got filtered out, was actually correct. The recall of a model is the proportion of true predictions compared to all instances that actually have that label. How much of what should have been filtered out, was actually filtered out. The F-score is the harmonic mean of precision and recall, providing a single metric to evaluate the model's performance. This takes both precision and recall into account, and balances them. A scenario where high precision is most important is your spam email filter. We all hate when we missed an important email because it got filtered as spam. When you would have a high precision, everything that is in the spam folder is actually spam. A scenario where high recall is most important is in medical dermatology diagnosis. It is worse to miss a cancer diagnosis (false negative) than to have a false alarm (false positive). These are the classic examples that were also used in previous courses I followed.

3.2. Code for confusion matrix and metrics

I already implemented the code to calculate recall, precision and F1-score. There is also a way to plot the confusion matrix in there.

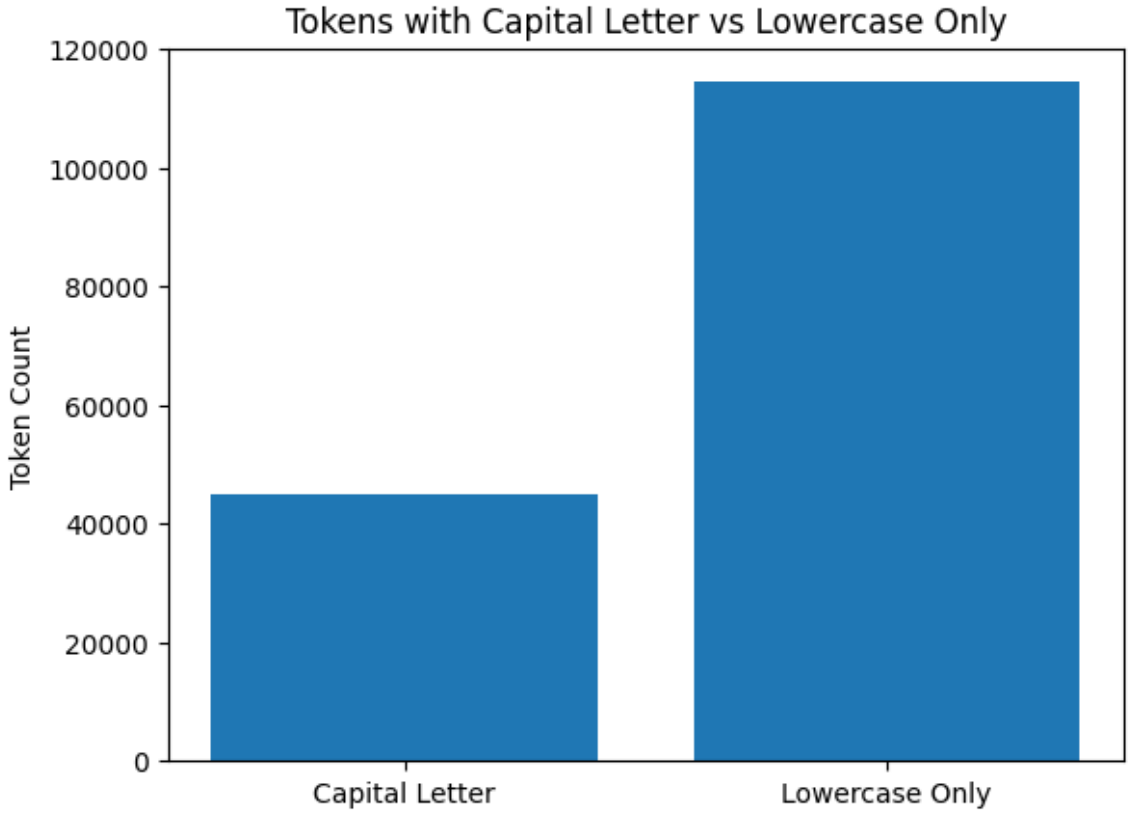

Figure 5: Capitalization distribution in the CoNLL training set.

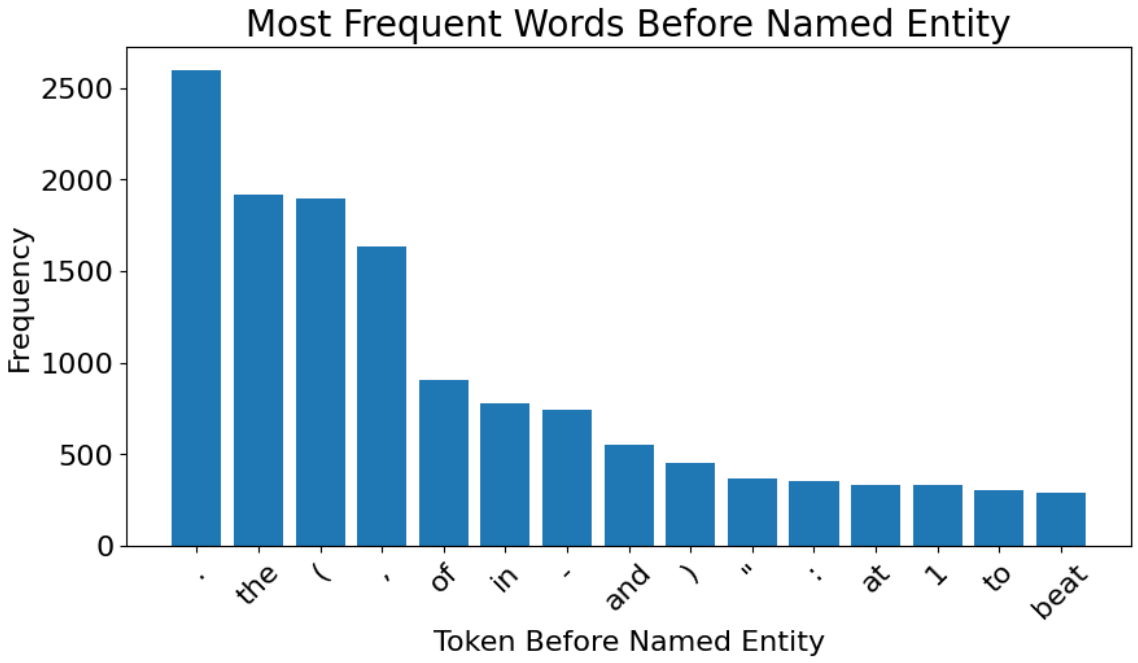

Figure 6: Top 15 most common tokens before named entities in the CoNLL training set.

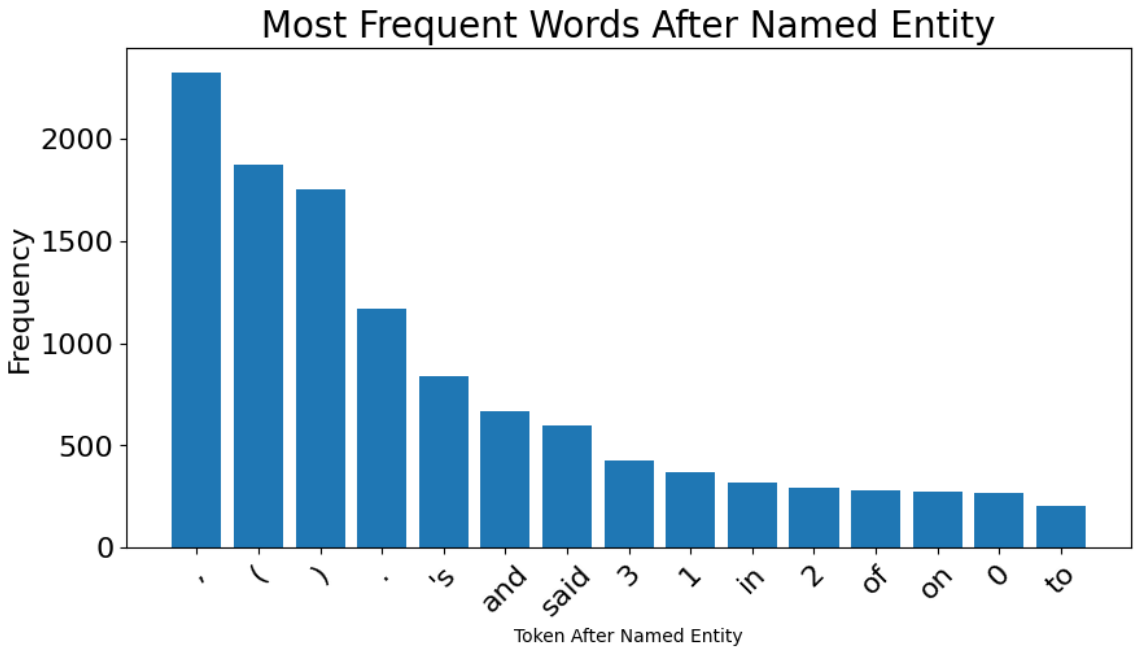

Figure 7: Top 15 most common tokens preceding named entities in the CoNLL training set.

3.3. Discuss token level evaluation and other related questions

In token-level evaluation, each token in the text is treated as an individual instance for evaluation purposes. In the case of the CoNLL dataset, each token is assigned a label indicating whether it is part of a named entity or not. And each datapoint in our dataset is a token. The advantages of analyzing at token level are:

- Granularity: Token-level evaluation provides a more fine-grained analysis of model performance, allowing us to identify specific tokens that are misclassified.
- Simplicity: Token-level evaluation is straightforward to implement and interpret, as it directly compares predicted labels to true labels for each token.

However, there are also disadvantages to token-level evaluation:

- Overestimation of performance: correctly classifying individual tokens may not necessarily translate to correctly identifying entire named entities, leading to an overestimation of model performance.
- Not like the real-world: In the real world, complete named entities are more important than individual tokens.

The advantages of using span-level evaluation instead of token-level evaluation are:

- More realistic: Span-level evaluation better reflects the real-world application of NER (it looks at the enire NE instead of individual tokens).
- Better assessment of entity recognition: Span-level evaluation provides a more accurate assessment of a model's ability to recognize complete named entities.

Since there are BIO tags present in the CoNLL dataset, it is possible to convert token-level predictions to span-level predictions. This can be done by grouping consecutive tokens with the same named entity label into spans. The spans will then be evaluated based on whether the entire span is correctly identified and classified as a named entity. No, BIO tagging is not strictly necessary for span-level evaluation, but it does facilitate the conversion from token-level to span-level predictions.

To deal with partial matches in span-level evaluation, we would need to use the strict evaluation approach. In this approach, a predicted span is considered correct only if it exactly matches the true span

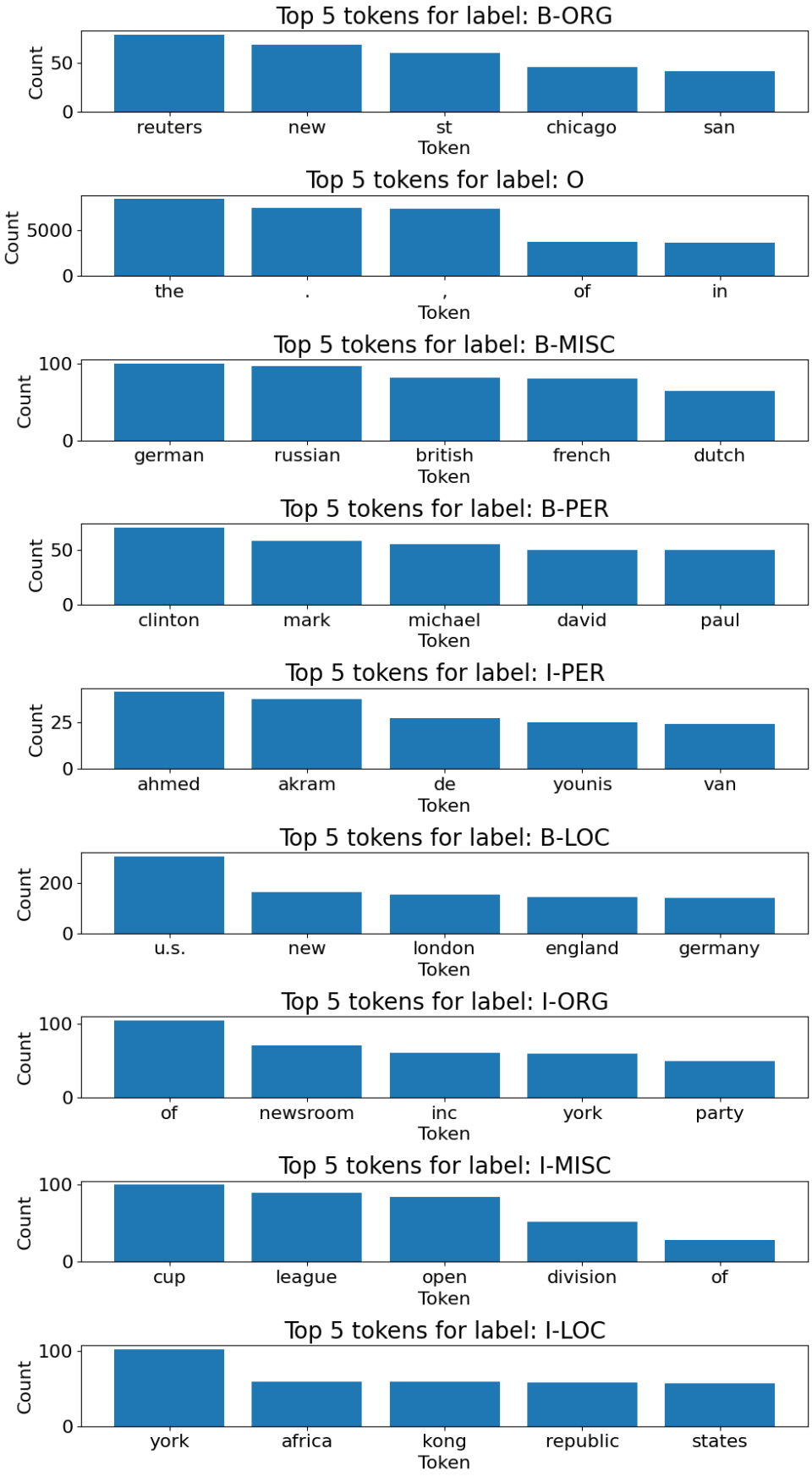

Figure 8: Most common words per named entity label in the CoNLL training set.

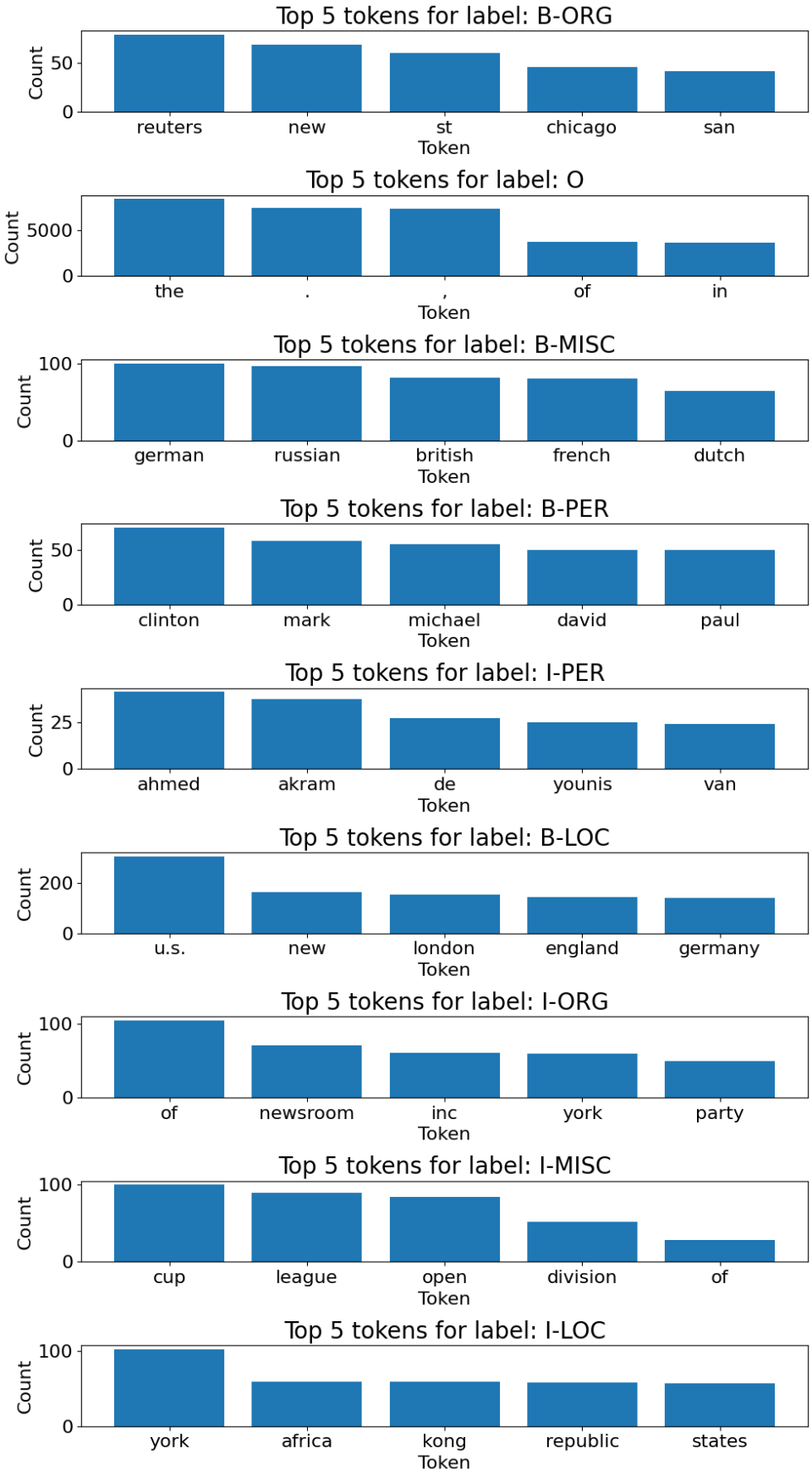

Figure 9: Most common words per named entity label (without IOB) in the CoNLL training set.

in both boundaries and label. This means that if a predicted span overlaps with the true span but does not exactly match it, it is considered incorrect. This strict evaluation approach ensures that the model is evaluated based on its ability to accurately identify and classify complete named entities.

Since the option of evaluating at span-level is advanced, I will focus on token-level evaluation for the assignments. This means, that the BIO tags will be taken into account when calculating the metrics and have to match exactly. If only a partial match is made (so either wrong BIO tag, or wrong NE label), it will be considered incorrect. In the notebook, I already implemented this evaluation function.

Assignment 2

New features from literature study After conducting a literature study on easy-to-implement features for NER, I found several features that could potentially improve the performance of our NER model. These features include:

- previous POS tag
- presence of digits
- position in the sentence

(Warto et al., 2024) provides a comprehensive overview of various features used in NER tasks. From the figure on page 920 in the paper, I concluded that the most used features were word embeddings, POS-tagging and character embeddings. Since word embeddings are part of the next step, I focused on the POS-tagging and the preceding POS tag. In (Nadeau and Sekine, 2007), the authors discuss various features for NER, including word features and lexical features. The table on page 8 in the paper shows several word-level features like the presence of digit and the position of the token in the sentence.

I implemented the features above in the code and will evaluate their impact on the model's performance in the next section.

Results of new features

In this section, I will present the results of incorporating the new features identified from the literature study into our NER model. I used logistic regression, SVM and Naive Bayes as classifiers to evaluate the impact of these features on model performance. The results are summarized in the following tables and confusion matrices: 1, 3, 3, 10, 5, and 12.

Table 3: NER classification report for SVM (not tuned hyperparameters)

Tag	precision	recall	f1-score	support
B-LOC	0.540	0.469	0.502	1837
B-MISC	0.658	0.482	0.556	922
B-ORG	0.322	0.406	0.359	1341
B-PER	0.489	0.558	0.521	1842
I-LOC	0.000	0.000	0.000	257
I-MISC	0.000	0.000	0.000	346
I-ORG	0.307	0.105	0.157	751
I-PER	0.453	0.826	0.585	1307
O	0.980	0.978	0.979	42759
accuracy			0.893	51362
macro avg	0.417	0.425	0.407	51362
weighted avg	0.889	0.893	0.888	51362

After hyperparameter tuning, the SVM model's performance improved significantly. Especially on the I-MISC and I-LOC labels, where previously no correct predictions were made. The results are shown in the table below 4 and the confusion matrix

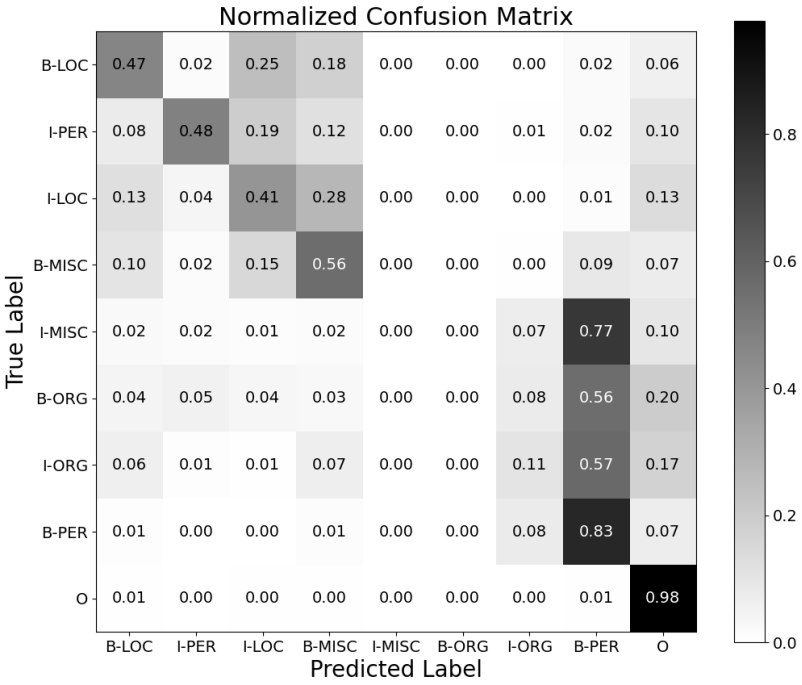

Figure 10: Confusion matrix for SVM model (not tuned hyperparameters).

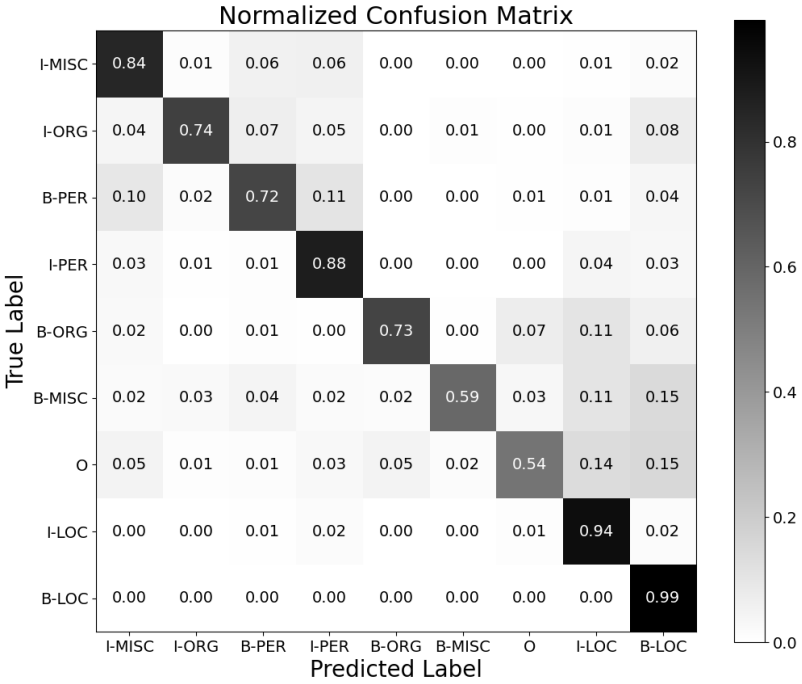

Figure 11: Confusion matrix for SVM tuned.

Table 4: NER classification report for SVM (tuned hyperparameters)

Tag	precision	recall	f1-score	support
B-LOC	0.841	0.843	0.842	1837
B-MISC	0.876	0.743	0.804	922
B-ORG	0.788	0.723	0.754	1341
B-PER	0.799	0.879	0.837	1842
I-LOC	0.796	0.728	0.760	257
I-MISC	0.839	0.587	0.690	346
I-ORG	0.834	0.543	0.658	751
I-PER	0.784	0.943	0.856	1307
O	0.990	0.995	0.992	42759
accuracy			0.961	51362
macro avg	0.839	0.776	0.799	51362
weighted avg	0.961	0.961	0.960	51362

Table 5: NER classification report for Naive Bayes

Tag	precision	recall	f1-score	support
B-LOC	0.562	0.848	0.676	1837
B-MISC	0.888	0.456	0.602	922
B-ORG	0.433	0.666	0.525	1341
B-PER	0.732	0.736	0.734	1842
I-LOC	0.000	0.000	0.000	257
I-MISC	0.000	0.000	0.000	346
I-ORG	0.733	0.344	0.468	751
I-PER	0.536	0.913	0.676	1307
O	0.989	0.963	0.976	42759
accuracy			0.912	51362
macro avg	0.541	0.547	0.517	51362
weighted avg	0.921	0.912	0.911	51362

An interesting observation is that the "O" label gets predicted very well by all models, with precision and recall values above 0.96. This is likely due to the high frequency of the "O" label in the dataset, making it easier for the models to learn to predict it accurately. On the other hand, labels like I-LOC and I-MISC have very low precision and recall across all models, indicating that these labels are more challenging to predict accurately. This could be due to the lower frequency of these labels in the dataset, making it harder for the models to learn their patterns.

Naive Bayes is very bad in the I-MISC and I-LOC labels, since it never predicts them. Logistic Regression performs best overall, with the highest accuracy (0.957), followed by Naive Bayes (0.912) and SVM (0.893).

SVM with word embeddings

I trained my SVM model with word embeddings based on the GoogleNews-vectors-negative300 model. The results are shown in the table below 6 and confusion matrix 13. Something to note about this table, is that the performance on the "O" label is steeply dropped compared to the SVM without the Google word embeddings. This is likely due to the fact that the word embeddings are not well suited for the "O" label, which is a very common label in the dataset. The overall accuracy of the SVM with word embeddings is 0.937, which is higher than the SVM without word embeddings (0.893). Note, I used a linear kernel for

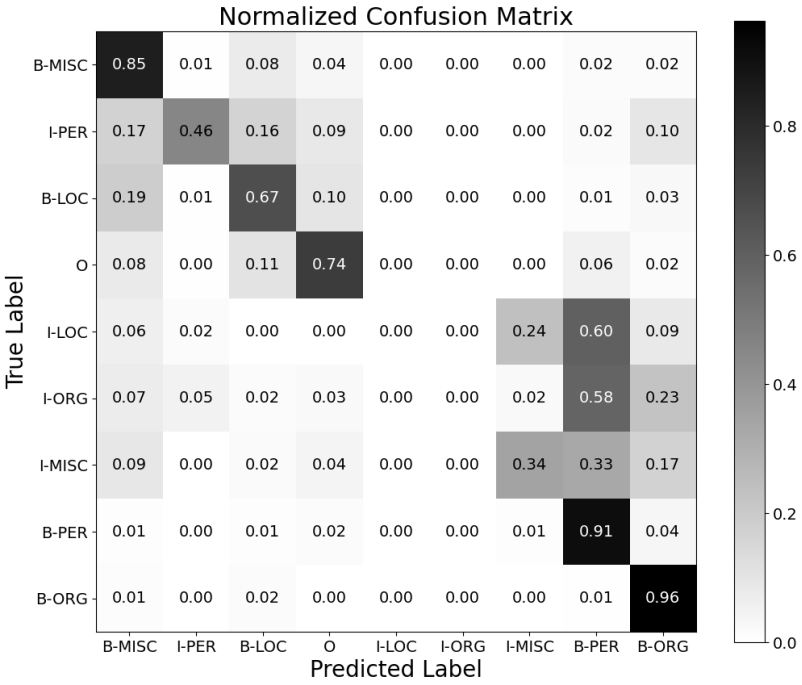

Figure 12: Confusion matrix for Naive Bayes model.

the SVM with word embeddings after the regular SVC took over 4h and still wasn't done training because of the large feature set.

Assignment 3

Feature ablation results To perform feature ablation, I created combinations of the features used in the (linear) SVM model. I picked the linear SVM since it performed quick and good in the previous assignment, and since I wanted to check a lot of combinations I needed something quick. The combinations that I picked are shown in table 7 along with their results. I created also confusion matrices for all options, but since there are so many I will not include them all here. In the figures folder for A3, they are all present, but in this report I will use only a few to illustrate the differences. Interestingly enough, the best performing model is the one with all features included. It is only a small difference though compared to the combination that doesn't use POS, digits and positions. From this we can conclude that the pos tag itself is not that important, but the preceding pos tag is. The confusion matrix for the best performing model is shown in figure 14.

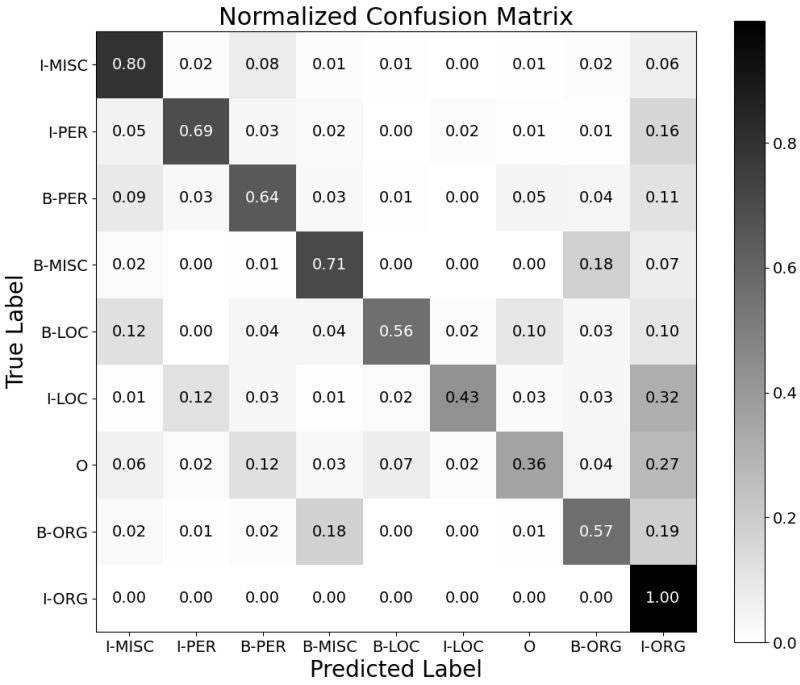

Figure 13: Confusion matrix for SVM model with word embeddings.

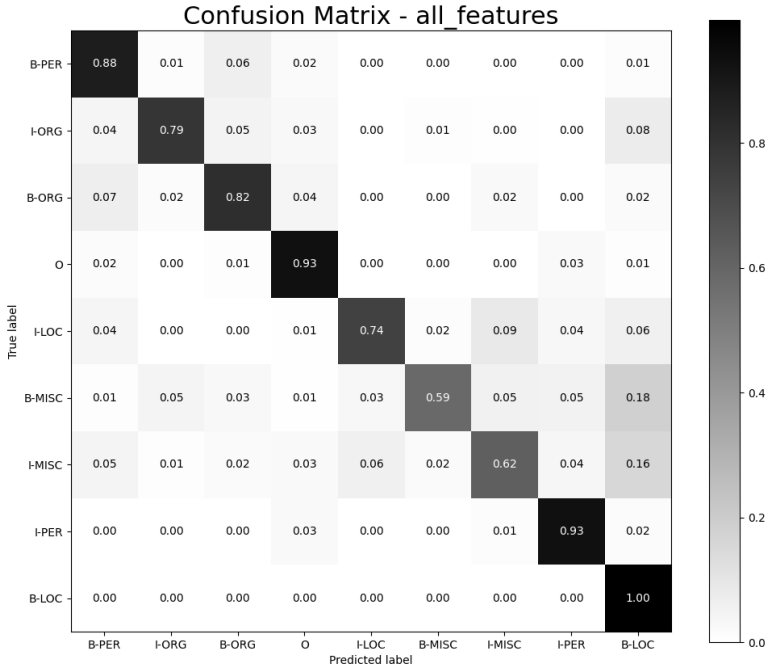

Figure 14: Confusion matrix for SVM model with all features.

Table 6: NER classification report for SVM with word embeddings

Tag	precision	recall	f1-score	support
B-LOC	0.818	0.795	0.807	1837
B-MISC	0.784	0.694	0.736	922
B-ORG	0.697	0.644	0.669	1341
B-PER	0.783	0.710	0.745	1842
I-LOC	0.608	0.560	0.583	257
I-MISC	0.677	0.431	0.527	346
I-ORG	0.610	0.361	0.454	751
I-PER	0.598	0.566	0.582	1307
O	0.974	0.996	0.985	42759
accuracy			0.937	51362
macro avg	0.728	0.640	0.676	51362
weighted avg	0.932	0.937	0.934	51362

Table 7: Feature Ablation Analysis Summary

Feature Combination	Active Features							Weighted F1	Weighted Recall	Weighted Precision
	emb	tok	pos	case	dig	posi	prev_p			
ALL_FEATURES	✓	✓	✓	✓	✓	✓	✓	**0.970**	**0.971**	**0.970**
EMBEDDINGS_TOKEN_PREV_POS_CASE	✓	✓	×	✓	×	×	✓	0.969	0.970	0.969
EMBEDDINGS_TOKEN_PREV_POS_DIGITS	✓	✓	×	×	✓	×	✓	0.968	0.969	0.967
EMBEDDINGS_TOKEN_PREV_POS_POS	✓	✓	✓	×	×	×	✓	0.967	0.968	0.967
EMBEDDINGS_TOKEN_PREV_POS	✓	✓	×	×	×	×	✓	0.965	0.966	0.965
EMBEDDINGS_TOKEN_PREV_POS_POSITION	✓	✓	×	×	×	✓	✓	0.965	0.966	0.965
ONLY_TRADITIONAL	×	✓	✓	✓	✓	✓	✓	0.961	0.962	0.962
NO_POSITION	✓	✓	✓	✓	✓	×	×	0.955	0.956	0.955
EMBEDDINGS_TOKEN_CASE	✓	✓	×	✓	×	×	×	0.953	0.954	0.954
EMBEDDINGS_PREV_POS	✓	×	×	×	×	×	✓	0.952	0.954	0.952
EMBEDDINGS_TOKEN_POS	✓	✓	✓	×	×	×	×	0.952	0.954	0.951
EMBEDDINGS_TOKEN	✓	✓	×	×	×	×	×	0.950	0.951	0.949
EMBEDDINGS_TOKEN_POSITION	✓	✓	×	×	×	✓	×	0.950	0.952	0.949
EMBEDDINGS_TOKEN_DIGITS	✓	✓	×	×	✓	×	×	0.950	0.952	0.949
EMBEDDINGS_POS_CASE	✓	×	✓	✓	×	×	×	0.945	0.947	0.945
EMBEDDINGS_POS_DIGITS	✓	×	✓	×	✓	×	×	0.942	0.944	0.942
EMBEDDINGS_CASE	✓	×	×	✓	×	×	×	0.941	0.942	0.942
EMBEDDINGS_POS	✓	×	✓	×	×	×	×	0.939	0.942	0.937
EMBEDDINGS_POS_POSITION	✓	×	✓	×	×	✓	×	0.939	0.942	0.938
EMBEDDINGS_DIGITS	✓	×	×	×	✓	×	×	0.934	0.938	0.933
EMBEDDINGS_POSITION	✓	×	×	×	×	✓	×	0.934	0.938	0.933
ONLY_EMBEDDINGS	✓	×	×	×	×	×	×	0.934	0.937	0.932
ONLY_MORPHOLOGICAL	×	×	✓	✓	✓	×	×	0.836	0.850	0.847